

Accelerating Sequential Circuit Simulation with Spatial Locality Enhancement and Redundant Event Reduction

Jiaping Tang^{*†‡}, Zizhen Liu^{*‡}, Jianan Mu^{*‡}, Feng Gu^{*‡}, Mingjun Wang^{*‡}, Wenxing Li^{*†},
Jing Ye^{*‡}, Xiaowei Li^{*†}, Huawei Li^{*‡}

^{*}State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China

[†]University of Chinese Academy of Sciences, Beijing, China

[‡]CASTEST Co., Ltd., Beijing, China

{tangjiaping22s, liuzizhen, mujianan, gufeng22s, wangmingjun22s, liwenxing21, yejing, lxw, lihuawei}@ict.ac.cn

Abstract—Fast simulation is vital for efficient digital design, especially for safety-critical applications, where functional safety verification is paramount. However, existing gate-level event-driven simulators often encounter performance challenges attributed not only to inefficient memory access, but also to redundancy events in sequential elements during event-driven algorithms. In this paper, we introduce a memory-efficient, low-redundancy event-driven simulation framework to accelerate sequential circuit simulation. Firstly, we propose an event-based memory layout approach that fully considers memory access characteristics within and between logic levels to enhance the spatial locality of simulators. Secondly, we present an event trace approach tailored for flip-flops to reduce event redundancies that hinder simulator performance. Comparative experiments demonstrate that our proposed optimization strategies deliver an average performance improvement of $1.9\times$ for logic simulation and $1.4\times$ for fault simulation.

Index Terms—simulation, memory layout optimization.

I. INTRODUCTION

Fast simulation is vital for efficient digital design, especially as modern processors and SoCs incorporate numerous complex components such as cores, accelerators, and memory hierarchies. Simulating these systems is essential to explore the design space and ensure correctness, with verification being a dominant factor in chip design cost. However, existing simulators are often slow, posing challenges in meeting tight time-to-market deadlines. This is particularly crucial in safety-critical applications, where functional safety verification is paramount [1]. At the core of this verification lies the fault injection or fault simulation campaign, which becomes more time-consuming as the number of faults that need to be simulated increases. Therefore, it is imperative to explore ways to accelerate simulation.

Simulators can be categorized along two dimensions. On the basis of scheduling algorithms, simulators can be divided into event-driven and oblivious simulators. Event-driven simulators perform evaluation on specific gates when changes occur in their inputs, which has the advantage of reducing unnecessary evaluations, especially when the design activity is low

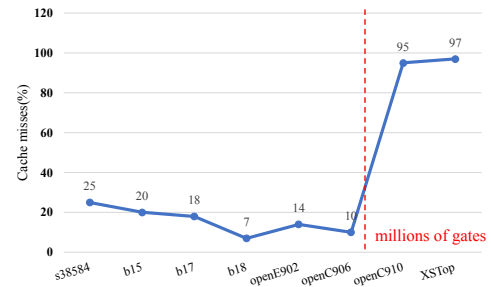


Fig. 1: Cache misses of logic simulation using a state-of-the-art commercial fault simulator [3].

[2]. From implementation techniques perspective, simulators can be classified as interpretive and compiled. Interpretive simulators excel in event-driven simulations due to their natural alignment with the event-driven paradigm and the simple execution flow. However, memory access remains a bottleneck for event-driven simulators, due to the fact that the memory access of gate-level event-driven simulation can be approximately viewed as a form of random memory access, leading to cache misses bottlenecks. Figure 1 illustrates the cache misses occurring during logic simulation by a state-of-the-art commercial fault simulator, Z01X [3], and the cache misses can exceed 90% for large designs, which applies to fault simulation as well. These cache misses primarily stem from two factors. The gate-level netlist is too large and is impossible to fully cache. Further, since the entire gate-level netlist is loaded at run-time and the gcc/g++ compiler lacks context information on simulation algorithms, for example the memory access characteristics of the event-driven algorithm, so it is impossible for the gcc/g++ compiler to perform memory layout optimization at compile time. Some existing work has focused on optimizing the memory access of oblivious simulators, such as reducing memory access instructions [4]. Nevertheless, optimizing event-driven simulators poses distinct challenges due to the dynamic loading of gate-level netlists for interpretive event-driven simulators and the gcc/g++ compiler lacks the run-time memory access characteristics for both

interpretive and compiled simulators.

In addition to memory access, the redundancy of events is another factor influencing performance for event-driven simulators. The number of events is one of the most critical factors affecting performance for event-driven simulators. There are many methods to decrease the number of events [5] [6]. However, these methods address only combinational logic, overlooking sequential elements that frequently trigger redundant events, as indicated by our test.

In this paper, we accelerate sequential circuit simulation by addressing both the memory access and computation aspects, aiming for the utmost reduction of redundant events and optimization of memory layout in event-driven simulators. For memory layout, we take full account of the execution characteristics of event-driven simulators to improve spatial locality. Additionally, to further reduce the number of redundancy events in event-driven simulators, we track sequential elements in the gate-level netlist to minimize unnecessary evaluations.

- We propose a memory layout method tailored for event-driven simulators to enhance the spatial locality of the data cache. Performance analysis using the perf tool on the benchmarks employed in this study shows a 40% reduction in cache accesses, a 20% decrease in cache misses rate, and a $1.6\times$ increase in IPC (Instructions Per Cycle).
- We optimize the simulation algorithm by selectively ignoring clock events for flip-flops to reduce redundant events for sequential elements. Experimental results for certain open-source designs demonstrate the reduction of events by up to 95% and nearly a $2\times$ performance improvement.
- Comparative experiments show that our optimization strategies can achieve $1.9\times$ performance gain for logic simulation and $1.4\times$ for fault simulation on average.

The remainder of this paper is organized as follows. Section II highlights the background. Section III describes the proposed simulation framework with memory layout optimization and redundant events reduction. Section IV empirically studies the effectiveness of our optimization schemes and the overall performance of our simulator. At last, Section V concludes the paper.

II. BACKGROUND

A. Related Works

Vertex reordering is a way to improve locality in graph computations. Given an input order, reordering aims to compute an alternate permutation of the vertices that is aimed at maximizing a locality-based objective [7]. There are different reordering strategies that have been used in practice, such as the window-based schemes [8] and the hub-based schemes [9]. They are intended to minimize cache misses and improve better spatial locality. However, these methods rely on the static structure of the graph itself and lack awareness of the memory access characteristics of specific algorithms such as the event-driven simulation algorithm.

B. Event-driven Simulator

Event-driven simulators perform an evaluation of specific gates when changes occur in their inputs, using events (value changes) to advance the entire simulation process. Events arise mainly from the evaluation of primary inputs or logic gates. Once events occur, they are propagated and evaluated until the entire circuit reaches a stable state, that is, no new events are generated. In situations characterized by low-activity designs, event-driven simulator exhibits notably efficient performance. Reducing the number of events is one way to improve the simulator performance [5] [6].

III. THE PROPOSED SIMULATION FRAMEWORK

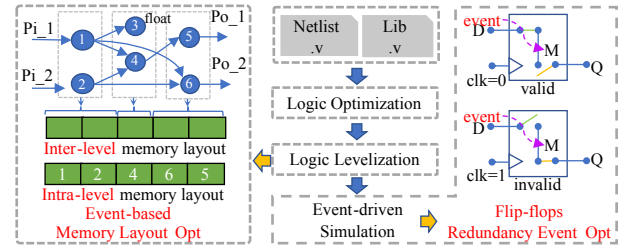


Fig. 2: The framework overview.

This section introduces the proposed memory-efficient, low-redundancy event-driven simulation framework, depicted schematically in Figure 2. We follow the basic workflow of simulators and propose two key optimization schemes to solve the memory access problem and the redundancy of events for event-driven simulators. First, we propose an event-based memory layout approach that fully considers the memory access characteristics at both the inter-level and intra-level for event-driven simulators. Second, we propose a event trace approach tailored for flip-flops to reduce the redundancy events that slash the performance of simulators.

A. Event-based Memory Layout

The event-based memory layout approach proposed to enhance spatial locality of event-driven simulators. The leveled circuit representation, as shown in Figure 3(a), is a highly effective method to reduce unnecessary evaluation of logic gates. When executing the event-driven simulation algorithm, events are always processed from lower levels to higher levels, which ensures that all fanins of a logic gate are in a stable state when performing its evaluation. Memory access is the primary cost for evaluation due to the fact that the computation of basic logic gates is tiny. In order to reduce the overhead of memory access, we propose an event-based memory layout approach to enhance spatial locality of event-driven simulators, which takes into account both inter-level and intra-level memory layout comprehensively.

Inter-level Memory Layout. As we all know, it is typically necessary to obtain basic information about the gate, such as fanins, fanouts, and gate type, when evaluating logic gates, which involves a large amount of memory access. If memory access is irregular, the spatial locality of the event-driven

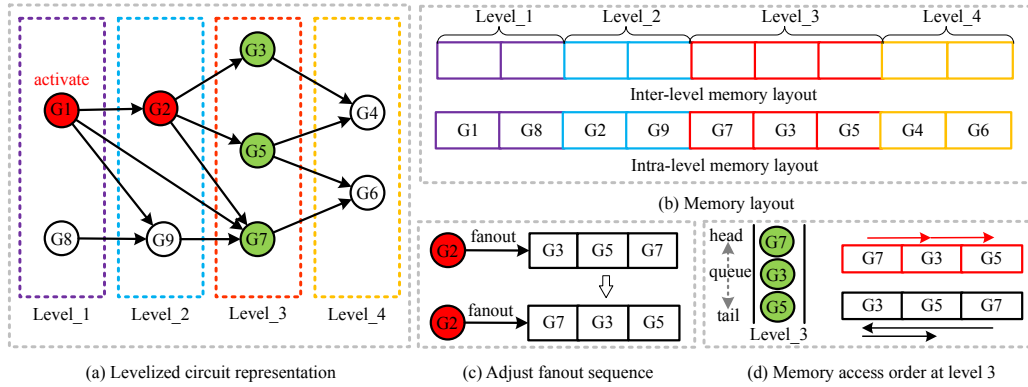


Fig. 3: (a) An example of levelized circuit representation. (b) shows the inter-level memory layout and intra-level memory layout of (a) using our event-based memory layout approach. (c) An example of adjusting the fanout sequence based on the memory layout shown in (b). (d) An example shows the memory access order of different memory layout approaches. Red arrows represent our method that is sequential access while black arrows is not.

simulators is suboptimal, which is one of the key factors impairing the performance of simulators. We find that memory access always comes from the lower level to the higher level during the execution of the event-driven simulation algorithm. So, we propose an approach that rebuilds the gate-level netlist and maps it to a contiguous memory space based on levels, as shown in Figure 3(b). By doing this, we can confirm that the memory access is always sequential between levels, which can reduce the random memory access across levels of event-driven simulators to some extent. Since the memory space is contiguous at each level, the spatial locality is better when executing the event-driven algorithm than that of the default memory layout, which depends on the gate-level netlist and the implementation of simulators.

Intra-level Memory Layout. The inter-level memory layout method described above only considers the storage of gates between levels, without considering the storage of gates within levels. So, there exists locally random memory access within each level during executing event-driven simulation algorithm. It is very beneficial to determine their access order to further reduce random memory access, for example, determining the order of gates at level 3 in Figure 3(a). The most perfect order is according to their scheduling order during executing event-driven simulation algorithm within each level. Pre-determining the scheduling order of each event within a level is a promising approach, but it requires sorting the events within the level, which may lead to significant performance overhead.

We propose a intra-level memory layout approach that makes a trade-off between sorting events within the level and absolute sequential memory access. The algorithm is based on the event-driven algorithm, as shown in Algorithm 1, which organizes the gate-level netlist according to the scheduled order of the event-driven algorithm. As shown in Algorithm 1 (lines 9-18), it executes the event-driven algorithm for the entire gate-level netlist, and the memory access of each gate is determined by their schedule order that is closely related to the implementation of scheduling algorithms, such as whether events are scheduled in a first-in-first-out or last-in-first-out

manner at each level. In this paper, we use first-in-first-out. After that, it adjusts the fanout sequence for each gate based on their order after inter-level and intra-level memory layout (lines 19-21). Consequently, the storage of gates within levels is always based on the order in which they are scheduled during the execution of the event-driven algorithm.

Figure 3(b) is obtained after applying inter-level and intra-level memory layout to Figure 3(a). Assuming G1 is currently active, then G2, G9, and G7 are added to the event queue. Subsequently, the evaluation of G2 leads to the addition of G3 and G5 to the event queue. Suppose that G9 does not generate events. Next, processing events at level 3, due to the queue order in level 3 being G7, G3 and G5 shown on the left of Figure 3(d), which align with our memory layout shown in the red part on the right side of Figure 3(d). So, the memory access is sequential, which is represented by red arrows using our proposed memory layout approach. If we follow the memory layout of G3, G5 and G7 shown in the black part on the right side of the Figure 3(d), then the memory access would not be sequential, which is represented by black arrows.

B. Redundant Event Reduction

To mitigate computational costs, we propose the redundant event reduction method, designed to identify and handle redundant events resulting from flip-flops. In event-driven simulation algorithm, a logic gate requires evaluation when any of its fanins experiences logic value change, which is also referred to as an event. However, there are certain special events that cannot cause a change at the output of logic gates, meaning that the gate evaluation yields the same value as before. We refer to these events that are impossible to cause value change at the output of a gate as invalid events. At least half of the evaluations are invalid in one clock cycle for flip-flops due to the fact that almost all flip-flops are working on a single edge. Given that the number of events significantly impacts the performance of event-driven simulators, identifying and handling these invalid events is a means to enhance efficiency.

Algorithm 1: Event-based Memory Layout Algorithm

Input : A gate-level netlist
Output: A rebuild gate-level netlist

```

1 schedule_queue  $\leftarrow$  an empty schedule queue;
2 memory_pos  $\leftarrow$  an empty vector that store gates;
  /* initial levelize queue */
3 foreach pi  $\in$  all primary inputs of netlist do
4   | push pi to schedule_queue;
5 end
6 foreach flip-flop  $\in$  all flip-flops of netlist do
7   | push flip-flop to schedule_queue;
8 end
9 while schedule_queue not empty do
  /* pop in level order */
10  | top_gate = pop a gate from the schedule_queue;
11  | push top_gate to the tail of memory_pos;
12  | foreach fanout  $\in$  top_gate's fanouts do
13    | if fanout has't been resolved then
14      | push fanout to schedule_queue;
15      | mark fanout has been resolved;
16    | end
17  | end
18 end
19 foreach gate  $\in$  all gates of netlist do
20   | Adjust_fanout_sequence(gate);
21 end

```

In order to quickly identify flip-flops that only involve clock events, for example dff1 in Figure 4(a), and whether there are valid events at current frame, we design a flip-flops marker queue, as shown in Figure 4(c), which stores all flip-flops that exist non-clock events to keep track of the valid events on flip-flops.

For ease of description, we model flip-flops as a Master-Slave model, which consists of two consecutive D latches, as shown in Figure 4(b). When $\text{clk}=0$, the Master captures the logic value at the D port, and when $\text{clk}=1$, the Slave captures the logic value from the Master. In this model, we define the following events as valid, and some cases are shown in Figure 4(d):

- If there is an event at the D port of the flip-flop and the current clock port is 0, then the event at the D port is considered valid. This is because the new logic value at the D port needs to propagate to the Master.
- If there is an event at the Master of the flip-flop and the current clock port is 1, then the event at the Master is considered valid. This is because the new logic value at the Master needs to propagate to the Q port.
- If there is an event at the set or reset port of the flip-flop, then the event is always considered valid. Here, we only consider asynchronous set or reset signals.

Apart from the aforementioned three scenarios, any events occurring under other circumstances are considered invalid

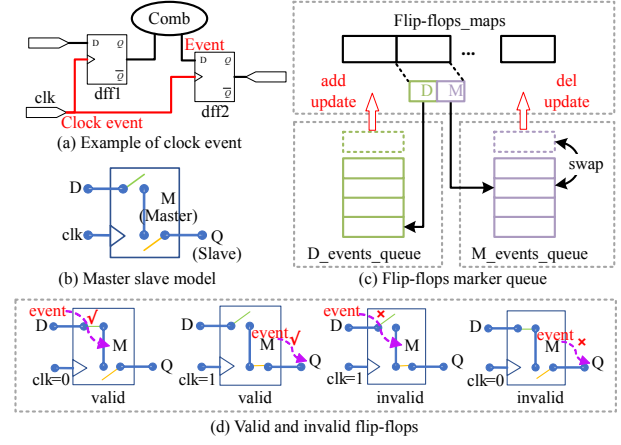


Fig. 4: (a) An example of clock event and dff1 can be neglected due to the existence of only clock events. (b) Master-Slave model that master captures value at D port when $\text{clk}=0$ and slave captures value from master when $\text{clk}=1$. (c) Three main components of our flip-flops marker queue. (d) Some examples of valid and invalid flip-flops.

events. If only invalid events exist on the flip-flop, it indicates that the flip-flop's evaluation will not generate new logic values. Therefore, we can ignore the flip-flop in the current frame. These invalid events should be stored in the flip-flops marker queue because they may either become valid events or disappear (e.g. $0 \rightarrow 1 \rightarrow 0$) in the future.

The flip-flops marker queue, illustrated in Figure 4(c), comprises three parts: D_events_queue, M_events_queue, and Flip-flops_maps. D_events_queue stores events occurring at the flip-flop's D port, while M_events_queue stores events occurring at the flip-flop's master latch. Flip-flops_maps stores all flip-flops containing non-clock events. When inserting a flip-flop into the flip-flops marker queue, we only push the event to either D_events_queue or M_events_queue based on the event type and then update Flip-flops_maps to establish connection relationships. Conversely, when removing a flip-flop from the flip-flops marker queue, we first swap the entry with the top entry and then adjust the connection relationships in Flip-flops_maps. If there are events at the set or reset port of flip-flops, we additionally augment a flag to the Flip-flops_maps.

After adding this flip-flops marker queue, there are two benefits. 1) If it only involves events in the clock signal at current frame, then we only need to process the flip-flops in the flip-flops marker queue, neglecting the handling of all flip-flops affected by the clock signal, for example dff1 can be neglected in Figure 4(a). 2) It helps us quickly identify whether there are valid events on flip-flops at current frame.

IV. EVALUATION

In this section, we assess the effectiveness of the proposed optimization strategies: event-based memory layout and flip-flops induced redundant event reduction, and conduct a comparative analysis of the overall performance against a baseline.

A. Experimental Settings

1) **Test Details:** We use gcc-11 to compile the simulator and enable the -O3 optimization option. We establish a baseline by implementing a simulator based on Proofs [13] and subsequently various engineering optimizations in [6], [12], without the optimization strategies proposed in this paper. We generate stuck-at faults on each cell port in the gate-level netlist for fault simulation. In addition, we set observation points to the primary output ports.

2) **Benchmarks:** The benchmarks used in our experiment are obtained from ISCAS89 [10], ITC99 [11], Pingtoug [14], and XiangShan processor [15]. Table I presents the benchmark used in this paper and the scale of each benchmark. After loading the netlist, logic optimization is used to preprocess the gate-level netlist.

TABLE I: Benchmark Information

Benchmark	# Number of logic gates		Optimization ratio%
	Initial	Logic Optimization	
s38584	16862	15353	9%
b15	12164	11708	4%
b17	37344	35902	4%
b18	103861	99975	4%
openE902	68566	63664	8%
openC906	624699	560058	11%
openC910	5324077	4714563	12%
XSTop	8618003	7062429	19%
Average			9.3%

B. Validity Evaluation

1) **Evaluate Event-based Memory Layout:** Table II illustrates the speedup achieved in logic simulation using the proposed event-based memory layout approach. The memory layout of the baseline is based on the sequence of logic gates in the netlist file. With the use of our optimization strategy, it is evident that the speedup for openC906, openC910, and XSTop can reach 1.3 $\times$, 1.4 $\times$, and 1.5 $\times$, respectively.

For further exploring the effect of our approach to CPU front-end bottleneck, we collect performance counter information using perf, which is a performance analysis tool. Our approach significantly impacts the data cache, prompting us to collect performance metrics specifically related to it. Figure 5 presents a comparison between our method and the baseline in terms of cache references, cache-misses rate, and IPC. As shown, the cache references have been reduced using our method by approximately 40% compared to the baseline. This reduction, to some extent, reflects better spatial locality. Additionally, it indicates that our event-based memory layout approach can effectively reuse values within registers, thus avoiding cache accesses. In terms of cache-misses rate, the cache-misses have been reduced by approximately 20% on circuits with millions of gates. This demonstrates that our method is cache-friendly, allowing better reuse of data within the cache and avoiding memory accesses to some extent, which is costly. Lastly, it can be observed that the IPC of our simulator has improved by 1.4 $\times$ and 1.6 $\times$ for the openC910 and XSTop benchmark. In general, our proposed event-based memory layout method effectively improves the spatial locality

of simulators, and since the optimization only requires a one-time setup with a time complexity of $O(n)$ (where n is the number of logic gates), the overhead is negligible, thereby enhancing simulation performance.

TABLE II: Time Comparison About Memory Optimization

Benchmark	Baseline(s)	Ours(s)	Speedup
s38584	8.1	7.4	1.1 $\times$
b15	2.4	2.4	1.0 $\times$
b17	13.8	11.5	1.2 $\times$
b18	33.8	28.1	1.2 $\times$
openE902	18.9	16.4	1.2 $\times$
openC906	200.7	147.8	1.3 $\times$
openC910	2232.5	1644.8	1.4 $\times$
XSTop	5236.2	3537.6	1.5 $\times$
Average			1.25 $\times$

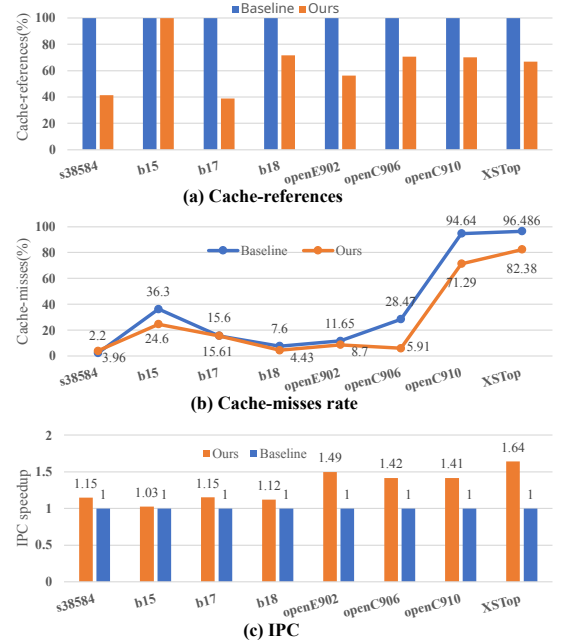


Fig. 5: (a) Our cache references relative to the baseline after using the proposed event-based memory layout approach. (b) The comparison of cache-misses rate. (c) The improvement of IPC.

TABLE III: Event Counts and Time Comparison About Redundant Event Reduction

Benchmark	Baseline*		Ours		Reduced	Speedup
	#event	time(s)	#event	time(s)		
s38584	148M	7.4	103M	6.9	30%	1.1 $\times$
b15	55M	2.4	38M	1.9	30%	1.3 $\times$
b17	112M	11.5	60M	6.9	47%	1.7 $\times$
b18	188M	28.1	67M	21.6	65%	1.3 $\times$
openE902	95M	16.4	22M	7.9	77%	2.1 $\times$
openC906	842M	147.8	93M	110.7	89%	1.3 $\times$
openC910	6,824M	1644.8	327M	1129.5	95%	1.5 $\times$
XSTop	16,138M	3537.6	1,027M	1775.5	94%	2.0 $\times$
Average					66%	1.5 $\times$

The Baseline*, under the promise of event-driven memory layout optimization.

2) *Evaluate Redundant Event Reduction*: Under the promise of event-driven memory layout optimization, we further evaluate the effectiveness of redundant event reduction optimization. Table III compares the event counts and time about the optimization of redundant event reduction for flip-flops. It can be seen that, with the use of our redundant event reduction optimization strategy, the total number of events when executing the event-driven simulation has reduced by an average of 66%. For openC910 and XSTop industrial designs, the event reduction is more remarkable, reaching up to 95% and 94%, respectively. In addition, for all benchmarks used in the experiment, the logic simulation performance has been improved by an average of $1.5\times$. More importantly, for industrial designs openC910 and XSTop with millions of gates, their performance has improved by $1.5\times$ and $2.0\times$ respectively.

However, we can find that there exists a certain gap between the acceleration ratio of the simulation and the proportion of event reduction. One major reason for this gap is that we continuously query valid flip-flops from the flip-flops marker queue in each frame. If there are many flip-flops in the marker queue, the querying overhead can be substantial. Another portion of the overhead comes from maintaining the flip-flops marker queue, including operations such as insert, delete, update, and query. We believe that this aspect can be further optimized to unleash greater optimization performance.

C. Overall Comparison

To evaluate the overall performance of the optimization strategies proposed in this paper, we apply them to both logic and fault simulations across all benchmarks. As shown in Figure 6, it is evident that the optimization strategies proposed in this paper result in significant performance enhancements for both logic and fault simulation, especially for the XSTop benchmark achieving a speedup of $3.0\times$ and $1.5\times$ respectively. On average, the logic simulation achieves a speedup of $1.9\times$, while the fault simulation realizes a $1.4\times$ speedup.

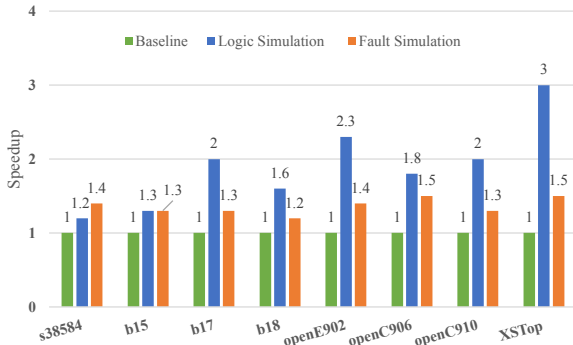


Fig. 6: The comparison of logic simulation and fault simulation.

V. CONCLUSION

In this paper, we present a low-redundancy, memory-efficient simulation framework to achieve the utmost reduction

of redundant events for sequential elements and optimize the memory layout to enhance spatial locality of simulators. For improving the spatial locality of the event-driven simulators, we propose an event-based memory layout approach that fully considers the memory access characteristics for event-driven simulators. For reducing the redundant events generated by sequential elements, we proposed an event trace approach tailored for flip-flops. Finally, the experimental results show that our methods can achieve a $1.9\times$ performance gain for logic simulation and $1.4\times$ for fault simulation on average.

ACKNOWLEDGMENT

This paper is supported in part by the National Natural Science Foundation of China (NSFC) under grant No. (62090024, 92373206, 92473203), and in part by the Chinese Academy of Sciences under grant No. XDB0660100. The corresponding authors are Zizhen Liu and Jing Ye.

REFERENCES

- [1] A. Nardi and A. Armato, "Functional safety methodologies for automotive applications," 2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Irvine, CA, USA, 2017, pp. 970-975.
- [2] Laung-Terng Wang, N. E. Hoover, E. H. Porter and J. J. Zasio, "SSIM: A Software Levelized Compiled-Code Simulator," 24th ACM/IEEE Design Automation Conference, Miami Beach, FL, USA, 1987, pp. 2-8.
- [3] Synopsys inc. [n. d.]. Z01X. <https://www.synopsys.com/verification/simulation/vc-z01x.html>.
- [4] K. Zhou, Y. Liang, Y. Lin, R. Wang and R. Huang, "Khronos: Fusing Memory Access for Improved Hardware RTL Simulation," 2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO), Toronto, ON, Canada, 2023, pp. 180-193.
- [5] E. M. Rudnick, T. M. Niermann and J. H. Patel, "Methods for reducing events in sequential circuit fault simulation," 1991 IEEE International Conference on Computer-Aided Design Digest of Technical Papers, Santa Clara, CA, USA, 1991, pp. 546-549.
- [6] P. M. Maurer, "The inversion algorithm for digital simulation," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 16, no. 7, pp. 762-769, July 1997.
- [7] R. Barik, M. Minutoli, M. Halappanavar, N. R. Tallent and A. Kalyanaraman, "Vertex Reordering for Real-World Graphs and Applications: An Empirical Evaluation," 2020 IEEE International Symposium on Workload Characterization (IISWC), Beijing, China, 2020, pp. 240-251.
- [8] Hao Wei, Jeffrey Xu Yu, Can Lu, and Xuemin Lin. 2016. Speedup Graph Processing by Graph Ordering. In Proceedings of the 2016 International Conference on Management of Data (SIGMOD '16). Association for Computing Machinery, New York, NY, USA, 1813-1828.
- [9] Zhang, Y., Kiriansky, V., Mendis, C., Zaharia, M.A., Amarasinghe, S.P. (2016). Optimizing Cache Performance for Graph Analytics. ArXiv, abs/1608.01362.
- [10] F. Brglez, D. Bryan and K. Kozminski, "Combinational profiles of sequential benchmark circuits," 1989 IEEE International Symposium on Circuits and Systems (ISCAS), Portland, OR, USA, 1989, pp. 1929-1934 vol.3.
- [11] S. Davidson, "ITC'99 Benchmark Circuits - Preliminary Results," International Test Conference 1999. Proceedings (IEEE Cat. No.99CH37034), Atlantic City, NJ, USA, 1999, pp. 1125-1125.
- [12] Hyung Ki Lee and Dong Sam Ha, "HOPE: an efficient parallel fault simulator for synchronous sequential circuits," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol. 15, no. 9, pp. 1048-1058, Sept. 1996.
- [13] T. M. Niermann, W. . -T. Cheng and J. H. Patel, "PROOFS: a fast, memory efficient sequential circuit fault simulator," 27th ACM/IEEE Design Automation Conference, Orlando, FL, USA, 1990, pp. 535-540.
- [14] head Semi inc. [n. d.]. OpenXuantie. <https://github.com/T-head-Semi>.
- [15] Y. Xu et al., "Towards Developing High Performance RISC-V Processors Using Agile Methodology," 2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO), Chicago, IL, USA, 2022, pp. 1178-1199.